Option A: Curling Game

Team Members:
Aaron Mai - aumai@ucsc.edu
- Github: aumai-ucsc

Leo Medellin - jlmedell@ucsc.edu
- Github: jlmedell

Link: https://github.com/jlmedell/Curling-Game

Controls:
Space (Hold) - Aim/Activate Power Slider
Space (Release) - Launch
A (While Holding Space) - Aim Left
D (While Holding Space) - Aim Right
Left Arrow (While Holding Space) - Adjust Curl
Right Arrow (While Holding Space) - Adjust Curl
R - Restart Game

Physics Summary: Two separate physics materials were defined, one for the ice and one for the stone prefab. They have similarly low friction and bounciness values, to ensure that the stones stay on the ice and slide easily across it. The stones are launched using AddForce on the Rigidbody to move them forward, with an AddTorque statement to apply spin and an additional AddForce block to apply force in a sideways direction, making it curl slightly as it nears the target. The Rigidbodies also have drag applied to ensure they gradually slow down, since the next stone isn't spawned until the game detects that the last has stopped, and collision detection is continuous on each stone's Capsule Collider, with the Y-position locked so that the stones can knock each other around and out of the scoring area but never fly off the ice. The last major physics feature is the trigger colliders for the scoring zone, which operate using OnTriggerEnter and OnTriggerExit. Whenever an object enters or leaves one of the scoring rings, these clauses detect it which facilitates the score calculation at the end of each round.

Aaron's Reflection:
I mainly worked on the scoring system for the curling game. Implementing things like colliders between the stones and the scoring zone, checking distance between stones, and scoring logic. I also implemented the restart button. I had a lot of trouble figuring out how to implement the colliders correctly to keep track of where the stones are on the map. I learned a lot about all three types of collision triggers, and a lot about how to use serialized fields and public variables to link different things between scripts. I eventually turned to checking if the stones were inside the home using bools and using Vector3Distance for overall scoring. I also got a lot of practice writing control flow. During testing, many bugs popped up where the game crashes while scoring. I learned that if there are null values in a tree of if statements, the null values have to be checked first, or else references to null gameObjects can cause issues.

Leo's Reflection: I did the initial setup in Unity, implementing features such as the aiming indicator, curving physics, low-friction materials, stone prefab and collision mechanics. I didn't have much experience with force application on 3D objects so this was a learning experience for me, but I'm proud of how everything turned out. The hardest part was the aiming indicator, which predicts the stones curling trajectory, updating in real time as the player rotates it. The vectors were a bit complex to work with, but for this reason it ended up being good practice.